Aula 08 - Exercício Prático: Busca Informada

Guilherme Bento Ramos - 185226

1. Fonte e Código Original

O código utilizado para a análise deste exercício é uma implementação didática do algoritmo A* para roteamento em grids 2D.

- **Autor:** Amit Patel (Red Blob Games)
- **Link original:** `https://www.redblobgames.com/pathfinding/a-star/`

O código de execução criado para gerar os cenários descritos neste relatório (`main.py`), bem como o módulo original do autor (`implementation.py`), estão anexados tanto nos arquivos de submissão quanto no final deste documento.

Análise do Algoritmo A* em Cenário de Armadilha

a) Execução do código e saída visual

O print abaixo mostra o nosso grid (25x10), onde 'A' é o ponto de partida e 'Z' o destino final. Os obstáculos ('###') foram posicionados para formar uma armadilha em "C"virada para a esquerda, bloqueando propositalmente o caminho direto em linha reta.

O trajeto escolhido pelo algoritmo é indicado pelas setas ($>$, $<$, $\wedge$, $\vee$), enquanto os nós que ele precisou visitar e analisar estão marcados com asteriscos ($*$). A execução mostra claramente que o algoritmo entrou na armadilha, explorou o interior e, ao ver que estava bloqueado, contornou a parede por cima. O custo total do caminho foi 26 e ele precisou explorar 151 nós para achar essa saída.

b) Como o algoritmo resolve o problema

O A* não avança às cegas; ele lida com o obstáculo de forma bem lógica e adaptativa. No começo, a busca vai direto para a direita, caindo na armadilha porque parecia ser a rota mais rápida. Quando ele bate na parede do fundo, o algoritmo explora os nós ali perto (dá para ver o acúmulo de asteriscos dentro do "C"). Ao esgotar as opções ali dentro e perceber que o custo estava ficando muito alto, ele recua e passa a focar nos nós ao redor que têm um custo total menor, o que permite que ele dê a volta por cima da barreira e continue até o destino.

c) Qual heurística foi aplicada?

O código usa a Distância de Manhattan. Basicamente, ela calcula a distância ignorando as paredes, como se estivéssemos andando em quarteirões livres. Funciona como um "ímã"apontando para o objetivo final. É exatamente por causa dela que o A* cai na armadilha logo de cara: a heurística o "puxa"com força na direção leste. Mas a verdadeira vantagem dela aparece no final: assim que o A* consegue contornar o obstáculo,

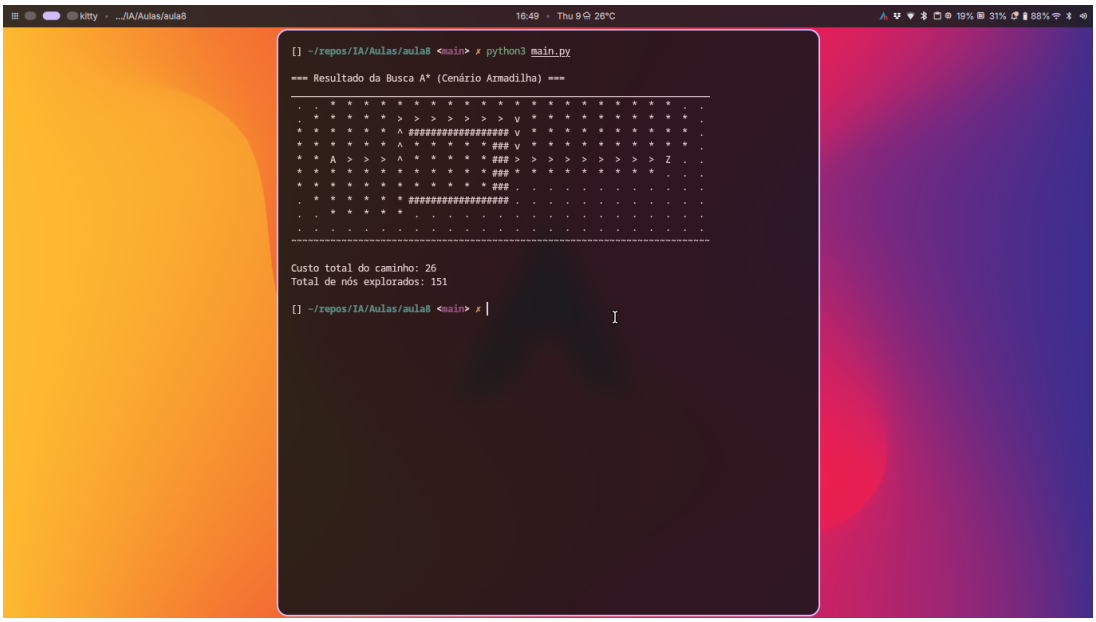

Figura 1: Saída do terminal mostrando a execução do A* lidando com a armadilha.

a heurística faz com que ele vá em linha reta até o 'Z', sem perder tempo explorando o resto do mapa à toa.

d) Comparação com buscas clássicas

Esse formato de armadilha é excelente para mostrar a diferença prática entre o A* e o algoritmo de Dijkstra (que é citado no material original). Se rodássemos o Dijkstra aqui, a busca iria se espalhar por todos os lados como uma onda na água, perdendo muito processamento explorando até a área atrás do ponto 'A', onde não há necessidade. O A*, guiado pela sua heurística, poupa recursos porque "sabe" a direção geral que deve seguir, explorando áreas vizinhas somente quando esbarra em uma parede e precisa achar um desvio.

Anexo: Códigos Fonte

Arquivo: main.py

```python
import implementation as imp

def custom_draw_tile(graph, id, style):
    r = "␣·␣"
    if 'explored' in style and id in style['explored']:
        r = "␣*␣"
    if 'path_dict' in style and id in style['path_dict']:
        next_node = style['path_dict'][id]
        if next_node:
            (x1, y1) = id
            (x2, y2) = next_node
            if x2 == x1 + 1: r = "␣>␣"
            if x2 == x1 - 1: r = "␣<␣"
            if y2 == y1 + 1: r = "␣v␣"
            if y2 == y1 - 1: r = "␣^␣"
    if 'start' in style and id == style['start']: r = "␣A␣"
    if 'goal' in style and id == style['goal']:   r = "␣Z␣"
    if id in graph.walls: r = "###"
    return r

imp.draw_tile = custom_draw_tile

def main():
    g = imp.GridWithWeights(25, 10)

    walls = []
    for y in range(2, 8): walls.append((12, y))
    for x in range(7, 13): walls.append((x, 2))
    for x in range(7, 13): walls.append((x, 7))
    g.walls = walls

    g.weights = {}

    start = (2, 4)
    goal = (22, 4)

    came_from, cost_so_far = imp.a_star_search(g, start, goal)
    path = imp.reconstruct_path(came_from, start, goal)

    path_dict = {}
    if path:
        for i in range(len(path) - 1):
            path_dict[path[i]] = path[i+1]
        path_dict[path[-1]] = None

    print("\n===␣Resultado␣da␣Busca␣A*␣(Cenario␣Armadilha)␣===")
    imp.draw_grid(g, explored=cost_so_far, path_dict=path_dict, start=start, goal=goal)
    print(f"\nCusto␣total␣do␣caminho:␣{cost_so_far.get(goal,␣'Caminho␣nao␣encontrado')}")
    print(f"Total␣de␣nos␣explorados:␣{len(cost_so_far)}\n")

if __name__ == '__main__':
    main()
```

Arquivo: implementation.py

```python
# Sample code from https://www.redblobgames.com/pathfinding/a-star/
# Copyright 2014 Red Blob Games <redblobgames@gmail.com>
#
# Feel free to use this code in your own projects, including commercial projects
# License: Apache v2.0 <http://www.apache.org/licenses/LICENSE-2.0.html>

from __future__ import annotations
# some of these types are deprecated: https://www.python.org/dev/peps/pep-0585/
from typing import Protocol, Iterator, Tuple, TypeVar, Optional
T = TypeVar('T')

Location = TypeVar('Location')
class Graph(Protocol):
    def neighbors(self, id: Location) -> list[Location]: pass

class SimpleGraph:
    def __init__(self):
        self.edges: dict[Location, list[Location]] = {}

    def neighbors(self, id: Location) -> list[Location]:
        return self.edges[id]

example_graph = SimpleGraph()
example_graph.edges = {
    'A': ['B'],
    'B': ['C'],
    'C': ['B', 'D', 'F'],
    'D': ['C', 'E'],
    'E': ['F'],
    'F': [],
}

import collections

class Queue:
    def __init__(self):
        self.elements = collections.deque()

    def empty(self) -> bool:
        return not self.elements

    def put(self, x: T):
        self.elements.append(x)

    def get(self) -> T:
        return self.elements.popleft()

# utility functions for dealing with square grids
def from_id_width(id, width):
    return (id % width, id // width)

def draw_tile(graph, id, style):
    r = " . "
    if 'number' in style and id in style['number']: r = " %-2d" % style['number'][id]
```

```python
    if 'point_to' in style and style['point_to'].get(id, None) is not None:
        (x1, y1) = id
        (x2, y2) = style['point_to'][id]
        if x2 == x1 + 1: r = " > "
        if x2 == x1 - 1: r = " < "
        if y2 == y1 + 1: r = " v "
        if y2 == y1 - 1: r = " ^ "
    if 'path' in style and id in style['path']:   r = " @ "
    if 'start' in style and id == style['start']: r = " A "
    if 'goal' in style and id == style['goal']:   r = " Z "
    if id in graph.walls: r = "###"
    return r

def draw_grid(graph, **style):
    print("___" * graph.width)
    for y in range(graph.height):
        for x in range(graph.width):
            print("%s" % draw_tile(graph, (x, y), style), end="")
        print()
    print("~~~" * graph.width)

# data from main article
DIAGRAM1_WALLS = [from_id_width(id, width=30) for id in [21,22,51,52,81,82,93,94,111,112,123,124,133,134,141,142,153,154,163,164,171,172

GridLocation = Tuple[int, int]

class SquareGrid:
    def __init__(self, width: int, height: int):
        self.width = width
        self.height = height
        self.walls: list[GridLocation] = []

    def in_bounds(self, id: GridLocation) -> bool:
        (x, y) = id
        return 0 <= x < self.width and 0 <= y < self.height

    def passable(self, id: GridLocation) -> bool:
        return id not in self.walls

    def neighbors(self, id: GridLocation) -> Iterator[GridLocation]:
        (x, y) = id
        neighbors = [(x+1, y), (x-1, y), (x, y-1), (x, y+1)] # E W N S
        # see "Ugly paths" section for an explanation:
        if (x + y) % 2 == 0: neighbors.reverse() # S N W E
        results = filter(self.in_bounds, neighbors)
        results = filter(self.passable, results)
        return results

class WeightedGraph(Graph):
    def cost(self, from_id: Location, to_id: Location) -> float: pass

class GridWithWeights(SquareGrid):
    def __init__(self, width: int, height: int):
        super().__init__(width, height)
        self.weights: dict[GridLocation, float] = {}
```

```python

    def cost(self, from_node: GridLocation, to_node: GridLocation) -> float:
        return self.weights.get(to_node, 1)

diagram4 = GridWithWeights(10, 10)
diagram4.walls = [(1, 7), (1, 8), (2, 7), (2, 8), (3, 7), (3, 8)]
diagram4.weights = {loc: 5 for loc in [(3, 4), (3, 5), (4, 1), (4, 2),
                                       (4, 3), (4, 4), (4, 5), (4, 6),
                                       (4, 7), (4, 8), (5, 1), (5, 2),
                                       (5, 3), (5, 4), (5, 5), (5, 6),
                                       (5, 7), (5, 8), (6, 2), (6, 3),
                                       (6, 4), (6, 5), (6, 6), (6, 7),
                                       (7, 3), (7, 4), (7, 5)]}

import heapq

class PriorityQueue:
    def __init__(self):
        self.elements: list[tuple[float, T]] = []

    def empty(self) -> bool:
        return not self.elements

    def put(self, item: T, priority: float):
        heapq.heappush(self.elements, (priority, item))

    def get(self) -> T:
        return heapq.heappop(self.elements)[1]

def dijkstra_search(graph: WeightedGraph, start: Location, goal: Location):
    frontier = PriorityQueue()
    frontier.put(start, 0)
    came_from: dict[Location, Optional[Location]] = {}
    cost_so_far: dict[Location, float] = {}
    came_from[start] = None
    cost_so_far[start] = 0

    while not frontier.empty():
        current: Location = frontier.get()

        if current == goal:
            break

        for next in graph.neighbors(current):
            new_cost = cost_so_far[current] + graph.cost(current, next)
            if next not in cost_so_far or new_cost < cost_so_far[next]:
                cost_so_far[next] = new_cost
                priority = new_cost
                frontier.put(next, priority)
                came_from[next] = current

    return came_from, cost_so_far

# thanks to @m1sp <Jaiden Mispy> for this simpler version of
# reconstruct_path that doesn't have duplicate entries

```

```python
def reconstruct_path(came_from: dict[Location, Location],
                     start: Location, goal: Location) -> list[Location]:

    current: Location = goal
    path: list[Location] = []
    if goal not in came_from: # no path was found
        return []
    while current != start:
        path.append(current)
        current = came_from[current]
    path.append(start) # optional
    path.reverse() # optional
    return path

diagram_nopath = GridWithWeights(10, 10)
diagram_nopath.walls = [(5, row) for row in range(10)]

def heuristic(a: GridLocation, b: GridLocation) -> float:
    (x1, y1) = a
    (x2, y2) = b
    return abs(x1 - x2) + abs(y1 - y2)

def a_star_search(graph: WeightedGraph, start: Location, goal: Location):
    frontier = PriorityQueue()
    frontier.put(start, 0)
    came_from: dict[Location, Optional[Location]] = {}
    cost_so_far: dict[Location, float] = {}
    came_from[start] = None
    cost_so_far[start] = 0

    while not frontier.empty():
        current: Location = frontier.get()

        if current == goal:
            break

        for next in graph.neighbors(current):
            new_cost = cost_so_far[current] + graph.cost(current, next)
            if next not in cost_so_far or new_cost < cost_so_far[next]:
                cost_so_far[next] = new_cost
                priority = new_cost + heuristic(next, goal)
                frontier.put(next, priority)
                came_from[next] = current

    return came_from, cost_so_far

def breadth_first_search(graph: Graph, start: Location, goal: Location):
    frontier = Queue()
    frontier.put(start)
    came_from: dict[Location, Optional[Location]] = {}
    came_from[start] = None

    while not frontier.empty():
        current: Location = frontier.get()

        if current == goal:
            break
```

```python

        for next in graph.neighbors(current):
            if next not in came_from:
                frontier.put(next)
                came_from[next] = current

    return came_from

class SquareGridNeighborOrder(SquareGrid):
    def neighbors(self, id):
        (x, y) = id
        neighbors = [(x + dx, y + dy) for (dx, dy) in self.
           NEIGHBOR_ORDER]
        results = filter(self.in_bounds, neighbors)
        results = filter(self.passable, results)
        return list(results)

def test_with_custom_order(neighbor_order):
    if neighbor_order:
        g = SquareGridNeighborOrder(30, 15)
        g.NEIGHBOR_ORDER = neighbor_order
    else:
        g = SquareGrid(30, 15)
    g.walls = DIAGRAM1_WALLS
    start, goal = (8, 7), (27, 2)
    came_from = breadth_first_search(g, start, goal)
    draw_grid(g, path=reconstruct_path(came_from, start=start, goal=goal
       ),
              point_to=came_from, start=start, goal=goal)

class GridWithAdjustedWeights(GridWithWeights):
    def cost(self, from_node, to_node):
        prev_cost = super().cost(from_node, to_node)
        nudge = 0
        (x1, y1) = from_node
        (x2, y2) = to_node
        if (x1 + y1) % 2 == 0 and x2 != x1: nudge = 1
        if (x1 + y1) % 2 == 1 and y2 != y1: nudge = 1
        return prev_cost + 0.001 * nudge
```